

Due: Saturday 10th October, 2026 at 23:59 on **Gradescope**.

A Unix-ish Shell (Part B)

In the first part of programming assignment (PA)-1, i.e., PA-1A, you created a simple shell by implementing four shell commands – `cat`, `grep`, `ls`, and `wc` – and wired them with a Read-Evaluate-Print Loop to run.

Now, in this second part (PA-1B), you will expand the existing implementation by (i) adding more commands (`head`, `tail`, `sort`, and `uniq`), (ii) running them as separate programs, (iii) connecting them into pipelines, (iv) managing running these command pipelines as background jobs, and (v) implementing built-in methods of `Shell.java` for facilitating `cd`, `jobs`, and `kill` commands along with redirection operator (`>` `<file>`). This document also explains Java language limitations and the workarounds adapted to mimic shell behavior.

You must finish reading this entire document to understand the full scope of this programming assignment.

Java Limitations. As we mentioned before, achieving the exact behavior of a Unix shell in Java is difficult without convoluted workarounds. This is because Java runs in the JVM (Java Virtual Machine), which does not provide direct access to some of the low-level process operations available on Unix systems. Below, we explain where these limitations arise and how we work around them in this assignment.

- (1) *Fork-exec*: A Unix shell uses `fork()` command to create a child process and `exec()` to run the command in that process. Java does not provide direct access to this mechanism. Instead, in this assignment, we use `ProcessBuilder` to start a new shell process and have it run the command. Unlike `fork()`, this new process does not inherit the Java state of the parent process.
- (2) *Pipes*: Similarly, a Unix shell creates OS-level pipes and connects the processes' inputs/outputs through them. Java, again, does not provide a way to create and attach these pipes in the same way. Therefore, to mimic this behavior, we use `ProcessBuilder.startPipeline()`, which creates the processes and connects their input and output streams for us.

Project Structure

Part B of *A Unix-ish Shell* project also consists of two modules, `commandBin` and `shell`. You will again start by implementing four new commands (`head`, `tail`, `sort`, and `uniq`) in the `commandBin` module. Each command is an independently executable program, has a `main` method, and extends `ShellCommand`.

The command's `main` method receives passed arguments, and the first argument is always the command name. Once the `main` method is called, we construct an instance of the command class before calling the instance's `runCommand` method, which, as its name suggests, runs the command's code.

SetUp

You can set up PA 1B in one of two ways, depending on your familiarity with git. The steps below are agnostic to the operating system (Windows, Linux, or macOS) you are using.

- (1) **Familiar with git** (`pull`, `push`, `checkout`, `merge` commands): Part B is published in a separate branch "PA1B" of the same repository. Run the following commands to include "PA1B".

*Note: This will not work if your git origin is pointing to your account git repository.
If that's the case for you, follow option (2) instead.*

```
git fetch origin PA1B
```

```
git merge origin/PA1B
```

Resolve any conflicts Git reports. `Shell.java` is the file most likely to conflict, since both you and Part B changed it — keep your own `findCommandClass` implementation. For `runRepl`, only the dispatch call changed, from `executeCommand(input)` to `processInput(input)`; the rest of the loop is the same one you already wrote. Skip next step (2) and go to "Did I successfully set it up?" section.

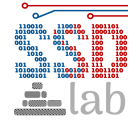
- (2) **Not familiar with git**: Follow the steps below.



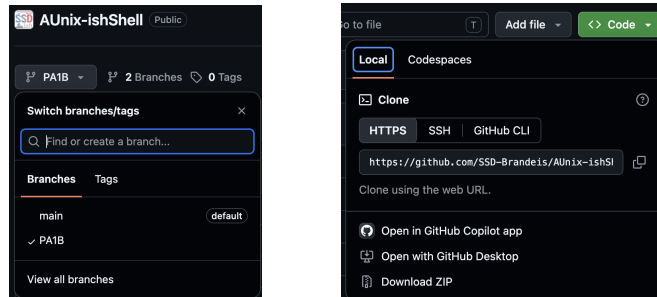
COSI 131A: Fundamentals of Computer Systems

Programming Assignment 1B

Fall 2026



- (a) First, **rename your PA 1A project folder** from `AUnix-ishShell` to `AUnix-ishShell-PA1A`, so your finished Part A work stays safe while you set up Part B of this programming assignment.
- (b) Download a fresh copy of `AUnix-ishShell-PA1B` project from GitHub (link). **Make sure you are on PA1B branch before you download a ZIP file**
Link: <https://github.com/SSD-Brandeis/AUnix-ishShell/tree/PA1B>



- (c) Now you need to copy your finished command code (`Ls.java`, `Grep.java`, `Wc.java`, `Cat.java`) to the fresh version you just downloaded. Locate the respective files in the Part A folder, which we renamed to "`AUnix-ishShell-PA1A/commandBin/src/main/java/`" and copy and paste the code to the new respective files in "`AUnix-ishShell-PA1B/commandBin/src/main/java/`". These commands don't change for Part B, so your Part A implementation works as-is.
- (d) At last, you need to copy two more functions' code from Part A of the project "`AUnix-ishShell-PA1A/shell/src/main/java/`". Open `Shell.java` (**Do not copy the entire file text**) and find the two functions below to copy their body (code between `{}`) and paste them into "`AUnix-ishShell-PA1B/shell/src/main/java/`".
- Copy your `findCommandClass` method over exactly as you wrote it.
 - Copy your `runRepl` method over too, then change the one line that calls `executeCommand(input)` so it calls `processInput(input)` instead. Everything else in the loop stays the same.

Did I successfully set it up?

To check if you successfully set it up, run `mvn clean` and then `mvn compile`. The output should look like below, where both `commandBin` and `shell` modules, as well as the overall project, should show **SUCCESS**.

```
[INFO] Reactor Summary for AUnix-ishShell 1.0-SNAPSHOT:
[INFO] AUnix-ishShell ..... SUCCESS [ 0.001 s]
[INFO] commandBin ..... SUCCESS [ 0.569 s]
[INFO] shell ..... SUCCESS [ 0.094 s]
[INFO] BUILD SUCCESS
[INFO] Total time: 0.720 s
[INFO] Finished at: 2026-09-23T11:21:51-04:00
[INFO]
```

Your Task

You should not have to change your implementation of the commands from Part A, but run the tests for `commandBin` before submitting, just to be safe. If you did not pass all the tests in the previous part, this is your chance to finish your implementation.

You must implement `runCommand` for each of the four new commands, following the same conventions as the commands you already wrote in PA-1A (extend `ShellCommand` and report errors to `System.err`). Below is a simplified description of each command. The tests are the source of truth for exact behavior.

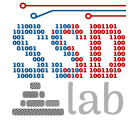
- `head [-n <count>] [<file>...]` - prints the first `count` lines (default 10) of each file passed as an argument. If more than one file is passed, each file's output is preceded by an `"==> <file> <=="` header line. If no files are passed, reads from standard input.
- `tail [-n <count>] [<file>...]` - prints the last `count` lines (default 10) of each file passed as an argument. Use the same header convention as `head` for multiple files. If no files are passed, reads from standard input.



COSI 131A: Fundamentals of Computer Systems

Programming Assignment 1B

Fall 2026



- `sort [-r] [<file>...]` - reads every line from all files passed as arguments (or from standard input if none are passed) and prints them sorted lexicographically — ascending by default, descending if `-r` is passed. Unlike `head` and `tail`, multiple files' lines are merged into one sorted output (no per-file headers).
- `uniq [-c] [<file>]` - collapses runs of adjacent duplicate lines in the given file (or standard input, if none is passed) into a single line; non-adjacent duplicates are left alone. If `-c` is passed, each output line is prefixed with the number of times it occurred in its run. The `uniq` takes at most one file argument.

Next, you will implement the following classes and methods in `Shell.java`. Use the comments in `Shell.java` and the tests to understand the required behavior for each method.

- class `Pipeline`
 - constructor - parses the command string (splitting on `|`), builds a forked process for each subcommand via `buildForkedShell`, and wires the first process's stdin and (unless already redirected to a file) the last process's stdout to the shell's own. A trailing `&` marks the pipeline as a background job.
 - `executePipeline` - starts every process in the current pipeline, each connected by real OS pipes via `ProcessBuilder#startPipeline`, and waits for them to finish unless the pipeline is a background job.
 - `waitFor` - waits for all processes in the pipeline to complete.
 - `killPipeline` - terminates all processes in the pipeline. `Process#destroy()` only requests termination asynchronously, so this also waits for each process to actually exit before returning.
 - `isDone`, `isBackground`, `isTerminated` - status checks for whether all processes have finished, whether the pipeline runs in the background, and whether it was terminated via `killPipeline`.
- Shell builtin methods (commands provided by the shell itself)
 - `changeDirectory` - (`cd`) updates the shell's working-directory. Note: you cannot change a running JVM's actual working directory; instead, the tracked directory is passed to every forked command.
 - `listJobs` - (`jobs`) prints each background job's id, status (`Running/Done/Terminated`), and original command. Job numbering does not restart unless the job list becomes empty.
 - `killJobs` - (`kill`) terminates the background job(s) with the given id(s).
- Shell methods for parsing and running commands
 - `runRepl` - runs the Read-Evaluate-Print Loop: print the prompt, read one line of input, and hand it to `processInput`. The command "`exit`" ends the loop.
 - `processInput` - parses one line of input to determine whether it names a shell builtin or a pipeline of separate processes, and executes it accordingly.
 - `tryExecuteBuiltin` - runs the input as a builtin (`cd/kill/jobs`) if it is one. Builtins run in the main shell process, not forked, since they mutate the shell's own state.
 - `buildForkedShell` - builds the `ProcessBuilder` that runs a single command as its own forked process, including detecting and setting up an output redirect (`> <file>`) if the command ends with one.

Testing

The assignment includes tests for both the shell commands (including new commands) and the shell itself. Refer to the README on how to set up and run these tests with *Maven*.

There are no hidden tests: if you pass the tests given to you, that means your program is working as intended. In fact, *you will need to use the tests as instructions for what behavior your implementation should have*. This assignment document describes the basic behavior of each part of the project code, but does not explicitly describe every detail. If you are failing a test, figure out what behavior is needed to pass the test; once you pass all the tests, your implementation is complete.



COSI 131A: Fundamentals of Computer Systems

Programming Assignment 1B

Fall 2026



Submission Guide

You must submit your assignment via Gradescope. Please ensure that all relevant project files are properly organized within a directory named `AUnix-ishShell-PA1B`. Your submission must include all necessary files so instructors can run and evaluate your project. For this assignment, you have two submission options:

1. Git Repository Submission:

Upload your folder to a GitHub or Bitbucket repository. Gradescope allows you to submit a link to your repository. Please ensure the repository is **public** or that you grant appropriate access permissions.

2. Zipped File Submission:

Alternatively, you can create a zip archive of your folder and submit it on Gradescope. Before zipping, remove all irrelevant files (such as temporary files or IDE-specific folders like `.vscode/` or `.DS_Store`).

The due date for this submission is Saturday 10th October, 2026 at 23:59. Late submissions may incur penalties as per course policies.

AI Log Submission

If you used an LLM while working on this Programming Assignment, submit a file named `LOG-PA1B.txt` (TXT, JSON, and Markdown files are acceptable) containing the full transcript of your interaction.

Link: <https://genai.cs.brandeis.edu/prompt/c254b018-90b4-470d-a461-5b14db9dec86>

Use the link above and include the following details.

- (1) Name of the LLM(s) used.
- (2) Every prompt you entered.
- (3) Every response you received.

You do not need to add labels such as “Question/Request” and “Answer/Response”. Save the conversation in a file as text, JSON, or Markdown.

All personal information is masked with random IDs and is not tracked anywhere, including email IDs.

Important Rules

- (1) Submit your prompts as-is verbatim. You do not need to paraphrase, summarize, or rewrite the prompts.
- (2) Please remove any personal information that is not relevant to the assignment, such as your name, username, email address, and any other personal information from file paths. Replace it with `<REDACTED>`.
- (3) If you haven’t used an LLM, please submit a file with saying: “No LLM assistance was used for PA”.

Using an **LLM is optional and will not affect your grade**. You are fully responsible for what you submit and should be able to explain it if and when asked to do so.